

# Kent Events

Technical Report

COMP6030



Sasha Ganhao  
**as2856@kent.ac.uk**

Samuel Passey  
**srgp2@kent.ac.uk**

Melanie Longden  
**kl447@kent.ac.uk**

## **Abstract**

Kent Events is a web-based event discovery and management platform developed for students, societies, and university-related organisers at the University of Kent. The project addresses the problem of fragmented event information across campus, where students may need to check several separate sources, including University event pages, Kent Union or society pages, and informal social channels such as Instagram or WhatsApp. This makes finding events inefficient and can cause students to miss relevant academic, social, or society-led activities. The submitted system provides a practical foundation for improving event visibility within the university environment. It supports core event discovery and management functionality, while also allowing for future extensions such as richer search, moderation, calendar integration, and wider automated event aggregation.

## **1. Introduction**

The aim of the project was to provide a more centralised interface through which events could be created, managed, browsed, and filtered. Rather than relying only on one institutional source, the platform was designed around a broader campus event model, supporting different account types and allowing both students and societies to contribute event information directly. This improves coverage by combining formal event listings with user-generated events, while also giving societies and individuals a clearer mechanism for promoting activities.

The system includes core functionality for account registration, authentication, email verification, event creation, event retrieval, filtering, event detail views, image handling, and automated event scraping. Scraped events are parsed and validated before being stored, ensuring that imported data conforms to the application's expected event structure. These features are supported by a React frontend, a Node.js backend, and a MySQL database. The system therefore focuses on delivering the essential workflow of a university event platform: users can register, verify their account, create or manage events where authorised, and browse available events through a consistent interface.

This report outlines the context and requirements that shaped the project before explaining the system architecture, implementation, development process, and key technical decisions. It then evaluates the maintainability of the submitted system, identifies realistic future work, and reflects on the strengths and weaknesses of the project and group process. A separate testing report accompanies it and provides the detailed test strategy, test environment, authentication and event test suites, iteration history, and testing limitations.

## 2. Related Work and Context

Existing event-discovery platforms demonstrate the value of centralising event information. Eventbrite allows organisers to manually create event listings with details such as title, date, time, location, event overview, and ticket information ([Eventbrite, n.d.](#)). This shows the usefulness of a structured event-listing model, but also highlights a limitation: coverage depends on organisers choosing to publish their events on that platform.

University event discovery presents a more specific version of this problem. The University of Kent provides official event pages with filtering by date, campus, category, and keyword, while KSU maintains a separate events page with its own search, student group, month, and category filters ([University of Kent, n.d.-a](#); [University of Kent, n.d.-b](#); [Kent Students' Union, n.d.-a](#)). KSU also describes student groups as organising regular meetups, social events, academic support, and trips, showing that student-led activity exists through union and society channels as well as official university pages ([Kent Students' Union, n.d.-b](#)).

Kent Events responds to this gap by adopting a hybrid model. Automated event collection allows events from existing sources to be brought into the platform, while manual event submission allows students and societies to create events directly. This differs from purely manual platforms by reducing reliance on organisers reposting the same event in another location, while still supporting user-made events. As a result, the project adapts common event-platform functionality to the specific context of university event discovery.

## 3. Requirements and scope

The scope of the project was to implement the core functionality required for a centralised university event platform. The system was designed to support event discovery, account-based event creation, filtering, authentication, email verification, automated scraping, and image handling. More advanced features, such as personalised recommendations, semantic search, calendar integration, and event analytics, were considered outside the scope of this version.

The platform uses four internal account types: student, society, KentUni, and KSU. However, only student and society accounts are exposed during registration. This reflects the intended user-facing model of the application: students and societies can create accounts, verify their email addresses, log in, browse events, and create events. The KentUni and KSU account types are reserved for internally sourced

events, such as those discovered by the scraper. This allows the system to represent different event sources without exposing those account types to ordinary users.

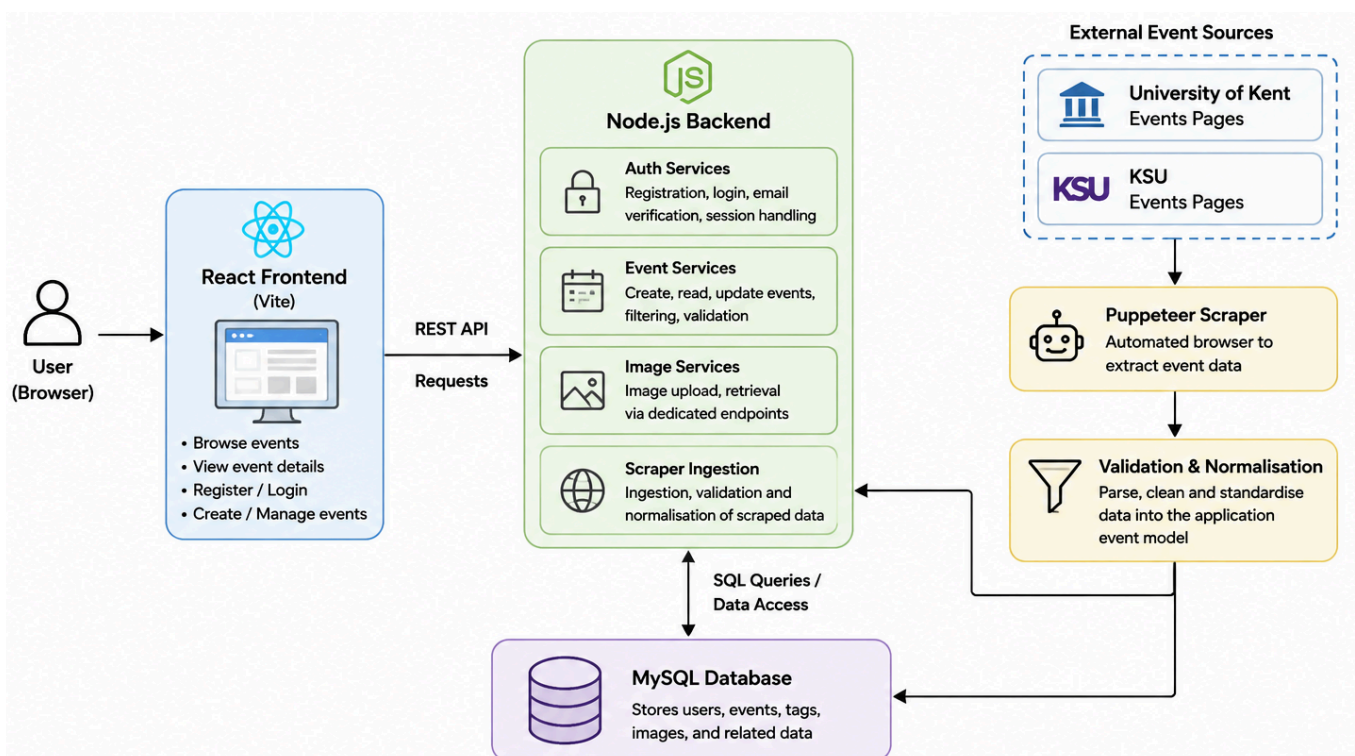
A key design decision was to derive an event's source from the account type of its owner rather than storing a separate source field in the event table. This reduces duplication and keeps source classification tied to ownership. For example, an event owned by a society account can be treated as a society event, while an event assigned to an internal KentUni or KSU account can be filtered accordingly. This design also keeps the event model simpler and avoids inconsistencies between an event's stored source and its actual owner.

Authentication is intentionally consistent across student and society accounts. The only difference during registration is the selected account type. After registration, users must verify their email address using a six-digit verification code before accessing the main functionality of the platform. This provides a lightweight verification layer while keeping the process appropriate for a student-focused web application.

Event functionality is built around a standardised data transfer object so that the backend, frontend, and tests handle event data consistently. Events include fields such as title, description, date, start time, end time, location, tags, contact details, ticket information, and image data. The browsing interface supports three main filters: date, source, and a string-based search. Image handling was included in the project scope so that events could include visual material without embedding raw image data directly in standard event list responses.

## 4. System architecture and design

The system was designed as a client-server web application with a React frontend, a Node.js backend, and a MySQL database. This separation allowed the user interface, application logic, and persistent data storage to be developed independently while communicating through a defined API contract. The frontend is responsible for presenting event information, handling user interactions, and sending requests to the backend. The backend manages authentication, session state, validation, event processing, image handling, scraping integration, and database access. The MySQL database stores user accounts, event records, verification data, tags, and image data.



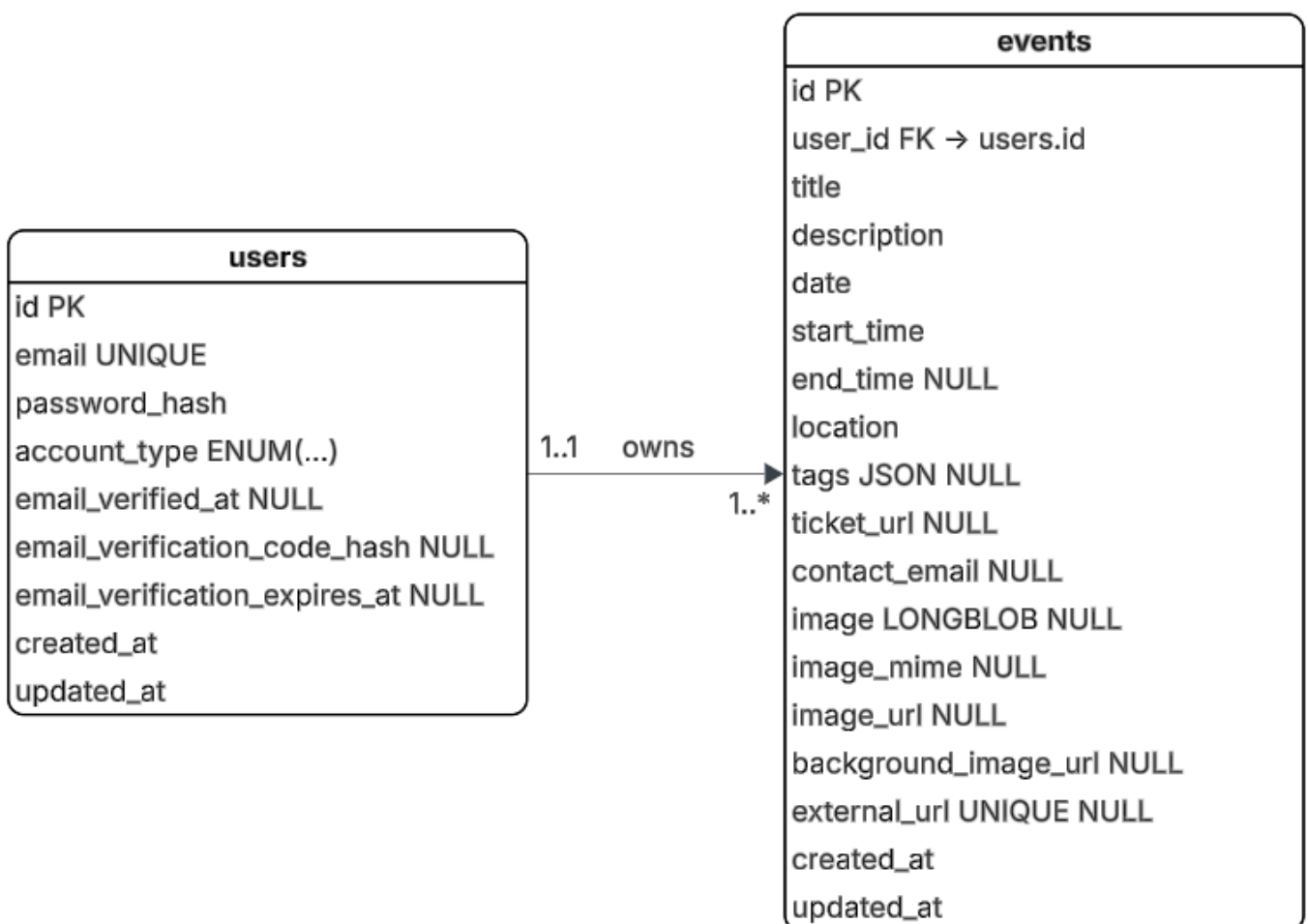
[Figure 1: High-level system architecture of Kent Events]

The frontend was implemented using React with Vite as the development and build tool. React was chosen because the application required dynamic user-facing pages, including registration, login, verification, event browsing, event detail views, and event creation or editing forms. Vite provided a lightweight development environment with fast rebuilds, which was useful during iterative frontend development. Styling was implemented using CSS, allowing the interface to be customised around the needs of the platform rather than relying entirely on a component library.

The backend was implemented using Node.js and organised around REST-style API endpoints. The API provides routes for authentication, email verification, event retrieval, event creation, event updates, filtering, and image retrieval. This structure

allowed the frontend to interact with the system through predictable request and response formats. Backend logic was separated into route, service, and repository-style responsibilities: routes handled HTTP-level concerns and validation, services handled application rules, and database access functions handled persistence. This separation improved maintainability by avoiding direct coupling between HTTP handling and SQL queries.

A central design decision was the use of standardised event data transfer objects. Event data is used across multiple parts of the system, including backend responses, frontend rendering, automated tests, and scraped event ingestion. Standardising fields such as title, description, date, start\_time, end\_time, location, tags, contact details, ticket information, and image references reduced inconsistency between layers. This was especially important because events could originate from either manual user submission or automated scraping. Both sources needed to be normalised into the same application-level event model before being displayed or filtered.



[Figure 2: Simplified database structure]

The account model also influenced the architecture. The system supports four internal account types: student, society, Kent Uni, and KSU, although only student and society accounts are exposed during registration. Student and society accounts follow the same authentication flow and have similar capabilities. The internal Kent Uni and KSU accounts are used to assign ownership to scraped events. Rather than storing a separate source field on each event, the event source is derived from the account type of the owning user. This keeps source classification consistent with ownership and supports source-based filtering without duplicating source data in the event table.

Event filtering is performed server-side using database queries rather than being handled entirely in the frontend. The browsing interface supports filtering by date and source, as well as text search. Performing this logic in the backend keeps the frontend simpler and avoids sending unnecessary event data to the client before filtering. It also allows filtering behaviour to remain consistent across different clients or future interfaces.

Image handling was designed separately from standard event retrieval. Instead of embedding raw binary image data directly into event list or detail responses, the backend maps events to image URLs where appropriate. Images are then served through a dedicated image endpoint. This reduces the size of standard JSON responses while keeping image delivery a separate responsibility. It also allows events without images to be handled explicitly without complicating the main event DTO. This consistency also supported automated backend testing, as event responses could be asserted against a stable structure across routes. The detailed testing strategy and results are discussed in the separate testing report.

Automated scraping was included as an ingestion mechanism for existing University and KSU event sources. Puppeteer is used to access external event pages, extract relevant event fields, follow detail links where necessary, and collect information such as descriptions, times, images, tags, and ticket links. Scraped fields are normalised and validated before being stored so that imported events conform to the same structure as manually submitted events. This design allows scraped and user-created events to coexist within the same database and be displayed through the same frontend views.

Docker and Docker Compose were used to support consistent development environments across the team. Containerising the frontend, backend, and database services reduced configuration differences between machines and made it easier to run the full system locally. This was particularly useful for a group project, where an environment mismatch can otherwise create avoidable integration issues.

## 5. Implementation

Authentication was implemented to control access to event creation and management, while still allowing unauthenticated users to browse publicly available events. Users can register, verify their email address, log in, and log out through backend API routes, with the frontend handling the corresponding forms and navigation flow. Passwords are hashed before storage, so plaintext credentials are never stored in the database. Once a user logs in successfully, their authenticated state is maintained using a server-side session containing the core identity fields needed elsewhere in the system: user ID, email address, and account type.

The platform exposes two registration types to users: student and society. These are the only account types available during registration, as they represent the users who can manually create and manage events. Internally, the system also supports KSU and university account types, which are used to associate imported or scraped events with their source without allowing ordinary users to register as those accounts. This allows the same ownership model to support both user-submitted events and internally managed event data.

Email verification was implemented as a required step after registration. When a user creates an account, the system generates a six-digit verification code and stores the relevant verification data alongside an expiry time. The user is then sent into the verification flow, where they must enter the code before the account is treated as fully verified. If an unverified user attempts to log in, the frontend routes them back to the verification form rather than allowing normal access. Society registration uses the same authentication and verification process, but assigns the society account type during account creation.

The core event management functionality was implemented through backend routes for creating, retrieving, updating, and deleting events. When a user submits an event form, the backend validates and processes the submitted data before inserting it into the database. The stored event is associated with the authenticated user, ensuring that every manually managed event is linked to a known account. This association is also used to enforce ownership rules during update and delete operations, preventing users from modifying or removing events they do not own.

Events can be retrieved either as part of a list or individually. The individual event route accepts an event ID and returns the corresponding event data if it exists, while invalid or missing IDs are handled explicitly so the frontend receives predictable responses. Event update functionality allows owners to modify existing event information, including text fields, date and time fields, tags, contact information, ticket links, and image state. The final event implementation uses a standardised data transfer object across the backend and frontend, with core fields including title,



description, date, start time, end time, location, tags, contact email, ticket URL, and image URL. This keeps frontend form state, backend route handling, database mapping, and tests aligned around the same conceptual event shape.

Event listing was implemented through a backend route that returns collections of events based on query parameters supplied by the frontend. Rather than loading all events into the browser and filtering them locally, filtering is performed on the server. This keeps the frontend simpler and gives the backend control over how filters are applied to database queries. The route supports pagination using limit and offset parameters, allowing the frontend to request a controlled number of events rather than receiving the full dataset at once. It also supports date-based filtering for categories such as upcoming and past events, and source filtering based on the account type associated with the event owner. For frontend filtering, KSU events are grouped with society events, while university events remain separate, reflecting the way KSU-sourced events often overlap with society activity.

The response format was designed to be predictable for the frontend. When no events match the current filters, the route returns a successful response with an empty array rather than treating the result as an error. This allows the frontend to render an empty state cleanly. Each listed event is also mapped to include an image URL where appropriate, rather than including raw binary image data in the event response. This keeps list responses suitable for rendering event cards while avoiding unnecessary payload size.

Image handling was implemented separately from ordinary event retrieval. Instead of embedding image data inside event objects, events expose an imageUrl field that points to a dedicated backend image route. If the requested event does not exist, the backend returns a not found response. If the event exists but has no image, the route returns a no-content response. If an image is present, the backend returns the image data with the appropriate content type so the browser can display it correctly. This distinction allows the frontend to treat missing images as a normal display case rather than as a failed event retrieval.

Image update behaviour was also implemented as part of event editing. When updating an event, the user can replace the existing image, remove it, or leave it unchanged. These three cases are handled explicitly to avoid accidental deletion or ambiguous update behaviour. The backend also rejects invalid combinations, such as requesting image removal while simultaneously uploading a replacement image, making the image update contract stricter and easier to test. The behaviour of these backend routes was validated through the API test suite, with the detailed test cases and iteration history discussed separately in the testing report.

The frontend was implemented in React and integrated with the backend API through forms, event listings, filtering controls, and authentication pages. Registration and

login pages submit user credentials to the backend, then route users according to their authentication and verification state. Event creation and editing forms were built around the standardised event DTO, reducing the translation required between frontend state and backend request bodies. Filtering controls send query parameters to the backend, request an updated event list when the selected filters change, and render empty result sets as a normal state. Overall, the frontend acts as a user-facing layer over the backend contract, connecting authentication, verification, event management, filtering, and image display through the implemented API routes.

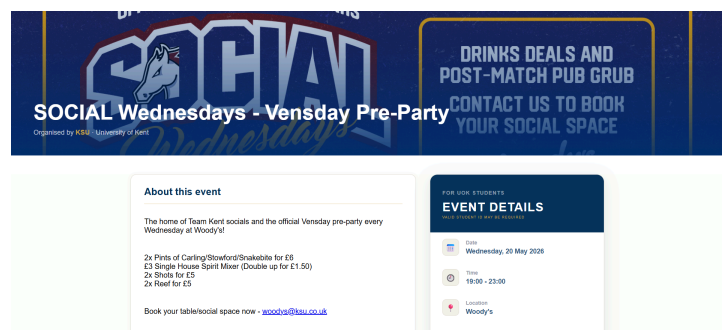
## 6. Development process and iteration

The development process for Kent Events was iterative, with the project moving from early feature construction into a later phase of refinement and stabilisation. In the initial stages, the priority was to establish the technical foundations of the platform. This involved learning and applying React, Node.js, Docker, Docker Compose, and MySQL, while also setting up the backend structure, database groundwork, authentication skeleton, and early frontend pages. Early development therefore focused on building working vertical slices of functionality, including registration, login, email verification, event creation, event editing, the homepage, and event details pages.

UI design was also part of the early development process. Several page layouts were drafted in Figma before implementation, including the homepage, event details page (Figure 3), sign-up page, login page, and create-event page. These designs provided an initial structure, but the interface changed as the platform became functional. During implementation, the homepage, event details page (Figure 4), and event cards were refined to improve spacing, visual hierarchy, use of event imagery, and the readability of key information such as titles, dates, times, locations, and sources.



[Figure 3: Original event details Figma design]



[Figure 4: Final event details page]

As the project progressed, the process became more structured around testing and maintainability. Automated backend testing was introduced first for the authentication system, before being expanded to the events functionality. This changed the development approach from simply implementing features to repeatedly validating route behaviour, edge cases, and API contracts. Testing also continued into later development, particularly after changes to authentication, event DTOs, filtering, search, and image handling. The backend had been designed from early development around separate route, service, and repository layers, but these layers were refined as the project matured. This made the boundaries between request handling, business logic, and database access clearer, making the system easier to test, reason about, and modify as requirements changed.

A major source of iteration came from the integration of scraped University of Kent and KSU events. Once scraped events were added to the same platform as manually created events, the homepage had to support a much larger and more varied dataset. This led to the introduction of pagination through a “Load More” model, where only an initial set of events was loaded before users requested additional batches. Filtering was also simplified during development, moving away from broader early ideas such as date ranges and price filtering towards a more focused set of source, date, and search controls that better matched the implemented event model. The final development phase focused heavily on stabilising the system’s data contracts. Several event fields were revised once the frontend, backend, database, scraper, and tests had to interoperate consistently. Time fields were standardised across the DTO, frontend, and tests, while weaker or redundant fields were removed or replaced. For example, `available_contact` was replaced with `contact_email`, and `price` was replaced with `ticket_url`, which better reflected the external nature of many university events. Image handling was also refined through image removal support, tag serialisation fixes, and more consistent event image behaviour.

Later iterations introduced account types and society registration, allowing event source filtering to be tied more closely to the owner of an event rather than to a separate event source field. The email verification flow was also rewritten to replace the earlier link-based approach with a clearer code-based user journey, including resend behaviour and handling for users who attempted to log in before verifying their email. Overall, development moved from establishing core functionality to repeatedly refining the platform in response to integration pressure, test feedback, and changes in the event data model. This produced clearer contracts, broader test coverage, and a system that better reflected how events were created, scraped, filtered, searched, and displayed.

## 7. Technical challenges and decisions

A key technical challenge was managing API contract instability as the event model evolved across the frontend, backend, database, and test suite. Several event fields changed during implementation as the data model became clearer. Earlier versions used less consistent field names such as `event_date`, `event_time`, and `end_event_time`, while the final event DTO standardised these as `date`, `start_time`, and `end_time`. Other fields were also refined or removed where they added ambiguity, with clearer alternatives such as `ticket_url` and `contact_email` replacing less useful fields. This standardisation mattered because the event DTO was used across API responses, frontend forms, event listings, and tests. Keeping the contract stable reduced the risk of frontend/backend mismatches and made route behaviour easier to validate through the backend test suite.

Image handling was another significant technical decision. Uploaded event images were stored in the database as BLOBs, meaning image files had to be converted into binary data before being saved. However, returning raw image data alongside every event would have made event list responses unnecessarily large, especially on pages such as the home page, where multiple events are loaded at once. To avoid this, the system separates event metadata from image retrieval. Event queries return the normal event fields and an image reference, while a dedicated image route retrieves the binary image only when it is needed by the frontend. This kept the main event responses lightweight and made the API behaviour clearer, as the image route could handle missing events, events with no image, and events with existing images separately.

The web scraping system also introduced several engineering challenges. Event information from the University of Kent and KSU websites was not presented in a single consistent structure, and some content was dynamically generated after the page loaded. The initial scraper used Cheerio, but this was unsuitable for pages where the relevant event content depended on JavaScript rendering. The implementation was therefore changed to Puppeteer, allowing the scraper to interact with pages after they had loaded in a browser-like environment. This improved reliability, but introduced a trade-off because Puppeteer is heavier and slower than static HTML parsing.

Further reliability work was required when scraping individual event detail pages. Some fields, such as descriptions, could be missed if extraction occurred before the relevant elements had loaded. Additional waiting logic was added before scraping continued, and validation checks were applied before database insertion. Events missing essential fields, such as titles or descriptions, were rejected, while less critical fields could remain empty. The scraper was also configured to run on a 24-hour cycle rather than continuously, reducing unnecessary load on both the Kent

Events system and the source websites. This introduced a limitation: existing events are not always rechecked for later changes on the original websites. Across these decisions, the priority was to keep the system predictable and maintainable, even where this meant accepting slower scraping, less real-time synchronisation, or more explicit API routes.

## **8. Maintainability and future work**

The Kent Events codebase was developed with maintainability in mind, especially as the project grew from a basic event platform into a larger system with authentication, account types, event management, image handling, filtering, scraping, and testing. A key maintainability decision was separating the React frontend, Node.js backend, and MySQL database. This made the system easier to reason about because interface behaviour, API behaviour, and database access were kept separate rather than being mixed up together. On the backend, the use of routes, services, repositories, DTOs, and validation helpers also helped keep responsibilities clearer, making individual areas easier to modify without affecting unrelated functionality.

Maintainability also improved as the event data model became more stable. Earlier versions of the project had inconsistencies between frontend fields, backend responses, database fields, and tests. Standardising the event DTO reduced this issue and made future changes easier to manage. Consistent naming, JSDoc documentation, and targeted comments were also used to make route behaviour and more complex implementation details easier to understand. GitLab supported this further by preserving a clear development history and making changes easier to track across the project lifecycle.

The backend test suite also supports maintainability because it acts as regression protection. The authentication and event tests validate important behaviours such as registration, login, email verification, session handling, event CRUD, ownership checks, filtering, image retrieval, and API response structures. This means that future backend changes can be checked against the existing expected behaviour, rather than relying only on manual testing.

However, the system still has limitations. The automated testing focuses mainly on backend API behaviour, while frontend rendering, browser-level interaction, accessibility, and performance under load were not tested to the same level. The use of a shared database for both the live application and tests also made isolation harder, even though unique test prefixes and cleanup functions helped reduce this issue. Image storage is another limitation. Storing uploaded images as database BLOBs worked for the prototype, but a production version would be better served by dedicated image or object storage, with the database storing only image references.

This would reduce database size, simplify backups, and make caching or CDN integration easier.

Realistic future work should therefore focus first on strengthening the existing platform. This would include introducing a dedicated test database, improving frontend and mobile responsiveness, and adding browser-level tests. The scraper system could also be improved through better monitoring, more robust failure handling, and support for additional sources, such as wider Canterbury events. Longer term, the platform could include admin moderation tools and personalised event recommendations, but these should come after the system has stronger production foundations and enough reliable usage data to make recommendations meaningful.

## **9. Critical reflection**

Overall, Kent Events was successful in delivering its central aim: bringing university event discovery into one place. The final system allows users to browse events publicly, while also supporting registration, login, email verification, society accounts, event creation, editing, deletion, filtering, image handling, and scraped event integration. As a result, the project developed beyond a simple event listing site into a fuller full-stack system involving authentication, ownership rules, database interaction, frontend/backend communication, and automated backend testing. Informal questionnaire feedback was limited, but the responses received suggested that the platform was generally easy to navigate. However, the small number of responses means this should be treated only as early feedback rather than strong user-evaluation evidence.

One of the main strengths of the project was the structure of the final system. Separating the React frontend, Node.js backend, and MySQL database made the application easier to understand, maintain, and extend. On the backend, routes, services, repositories, DTOs, and validation helpers helped keep responsibilities separate, which made individual behaviours easier to test and modify. The standardisation of the event DTO was particularly important, as earlier versions of the project had inconsistencies between frontend forms, backend responses, database fields, and tests. Stabilising this contract reduced the risk of frontend/backend mismatches. Separating image retrieval from event metadata was also effective, as returning raw image data in every event list response would have made pages unnecessarily heavy.

However, the project also had clear limitations. The scraping system remains dependent on the structure of external websites, meaning changes to the University of Kent or KSU event pages could break parts of the scraper. Image storage was

also functional rather than ideal. Storing uploaded images as database BLOBs worked for the scope of this project, but dedicated image hosting would be more flexible and more appropriate for a production system. Testing could also have gone further. The backend API tests gave useful confidence in authentication, event management, database side effects, session behaviour, and route responses, but the project did not have equivalent automated coverage for frontend rendering, browser-level interaction, accessibility, or performance under load. The shared database also made test isolation more difficult, although unique test data and targeted cleanup reduced the risk of interference.

The group process was productive overall, but it also showed the importance of making shared design decisions early in a collaborative project. GitLab feature branches and pull requests allowed work to be divided and merged without constantly overwriting each other's changes. Even so, some integration issues occurred where features overlapped or where shared assumptions had not been agreed in advance. For example, decisions around account types and society registration would have benefited from being finalised earlier, as these affected authentication, event ownership, and frontend behaviour.

With hindsight, we would define shared contracts earlier, especially the event DTO, account type model, and API response structures. We would also use a separate test database from the beginning, introduce frontend and accessibility testing earlier, and use dedicated image hosting rather than storing uploaded images directly in the database. More regular integration checkpoints would also have helped ensure that separately developed features remained aligned throughout development.

## **10. Conclusion**

Kent Events aimed to address the problem of fragmented event discovery at the University of Kent by providing a centralised platform for browsing, creating and managing events. The final system achieved this core aim by supporting public event browsing, account registration, authentication, email verification, event creation and management, filtering, image handling, and automated scraping from existing university event sources. The project also demonstrated the integration of a React + Vite frontend, Node.js backend, MySQL database, Docker environment, Puppeteer scraper, and Vitest-based backend testing within a full-stack web application.

While there are still areas for improvement, including mobile responsiveness, more advanced filtering, recommendation functionality, and wider deployment refinement, these are natural extensions rather than failures of the core system. Overall, Kent Events provides a strong foundation for a more complete university-focused events platform, while also giving the team valuable experience in collaborative full-stack development.

## **11. Acknowledgements**

We would like to acknowledge the contributions of all team members throughout the development of Kent Events, including work on the frontend, backend, database, testing, scraping, documentation, and final report. We would also like to thank our project supervisor for their guidance and feedback during the project. Additional thanks are given to those who provided questionnaire responses and informal feedback, as this helped inform the evaluation of the final system. The project also made use of official documentation for React, Node.js, MySQL, Docker, Vitest, and Puppeteer.



## References

Eventbrite. (n.d.) Create an event. Available at: <https://www.eventbrite.co.uk/help/en-gb/articles/551351/how-to-create-an-event/> (Accessed: 8 May 2026).

Kent Students' Union. (n.d.-a) Events. Available at: <https://ksu.co.uk/events> (Accessed: 9 May 2026).

Kent Students' Union. (n.d.-b) Activities. Available at: <https://ksu.co.uk/activities> (Accessed: 9 May 2026).

University of Kent. (n.d.-a) What's on. Available at: <https://www.kent.ac.uk/whats-on> (Accessed: 9 May 2026).

University of Kent. (n.d.-b) Student events. Available at: <https://student.kent.ac.uk/events> (Accessed: 9 May 2026).